

Informe: Contenedorizar la Aplicacion Spring Boot y Desplegar en Railway

Ingenieria de Sistemas — UDES 2026

1. Aplicacion Desplegada en Railway

La aplicacion Spring Boot del catalogo de productos fue desplegada exitosamente en Railway con una base de datos PostgreSQL conectada. La URL publica de acceso es:

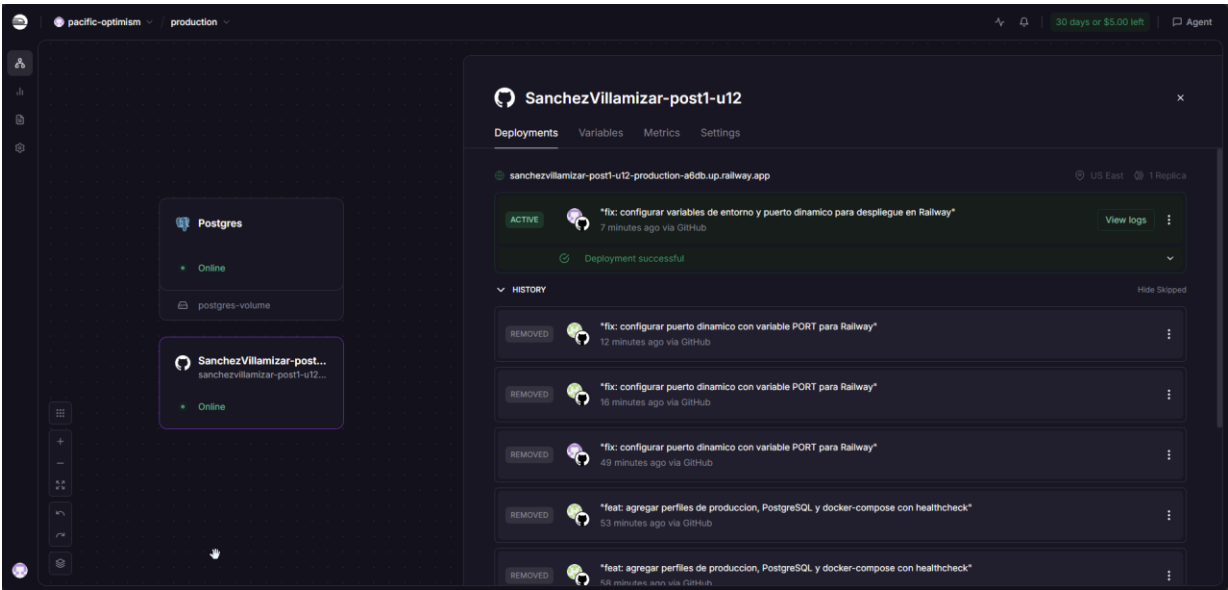
<https://sanchezvillamizar-post1-u12-production-a6db.up.railway.app>

Endpoints disponibles publicamente

Metodo	Endpoint	Descripcion	Status
GET	/api/productos	Lista todos los productos activos	200 OK
POST	/api/productos	Crea un nuevo producto	201 Created
GET	/api/productos/{id}	Busca un producto por ID	200 OK
DELETE	/api/productos/{id}	Elimina un producto	204 No Content
GET	/actuator/health	Estado de la aplicacion	200 OK

2. Panel de Railway

El proyecto en Railway consta de dos servicios desplegados: la aplicacion Spring Boot y la base de datos PostgreSQL. Ambos servicios se encuentran en estado activo y saludable.

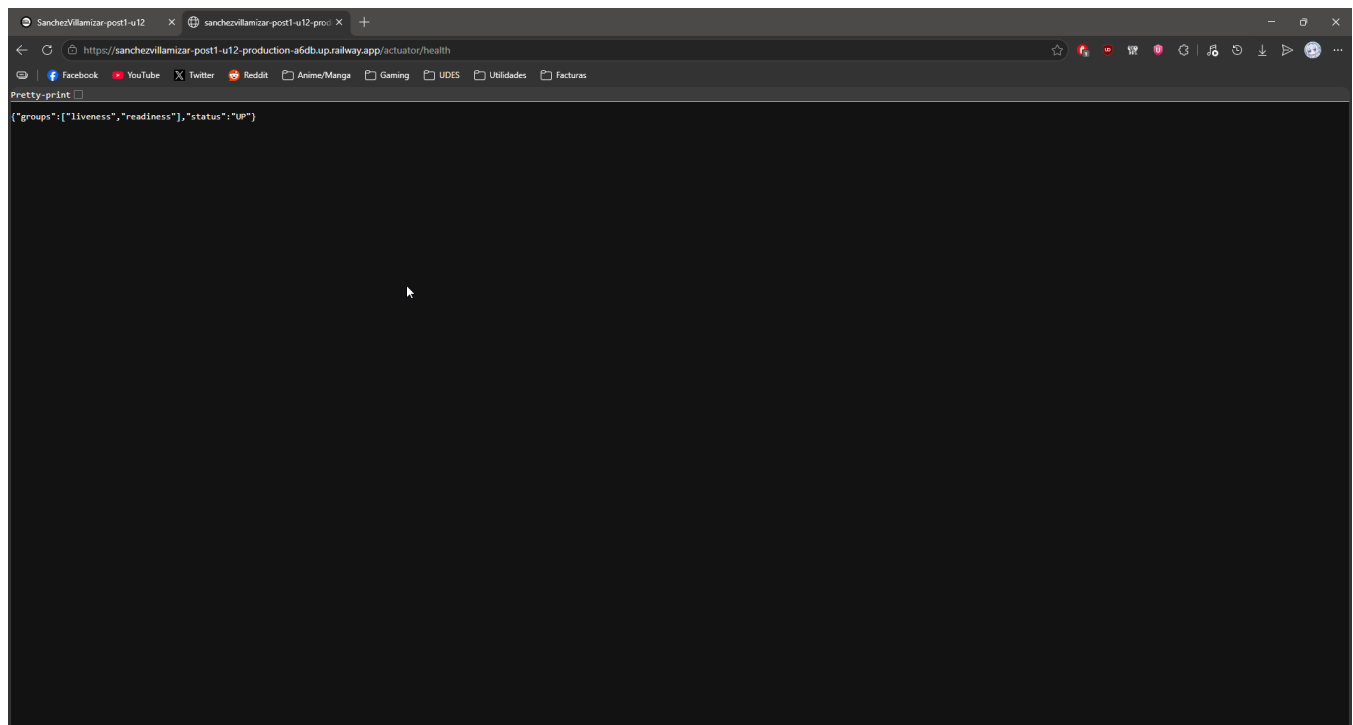


Servicios en Railway

Servicio	Tipo	Estado	Puerto interno
catalogo-app	Dockerfile (Spring Boot)	Online	8081
PostgreSQL	Database (postgres:16)	Healthy	5432

3. Healthcheck de la Aplicacion

El endpoint /actuador/health confirma que la aplicacion esta en linea y la base de datos PostgreSQL esta conectada correctamente:



Request

```
GET https://sanchezvillamizar-post1-u12-production-a6db.up.railway.app/actuador/health
```

Response (200 OK)

```
{"groups":["liveness","readiness"],"status":"UP"}
```

El status UP confirma que la aplicacion inicio correctamente con el perfil de produccion activo y la conexion a PostgreSQL establecida.

4. Endpoints Funcionando en Railway

Endpoint 1 — GET /api/productos

Lista todos los productos activos en la base de datos PostgreSQL:

```
GET https://sanchezvillamizar-post1-u12-production-a6db.up.railway.app/api/productos
```

Response (200 OK)

```
[ ]
```

Endpoint 2 — POST /api/productos

Crea un nuevo producto en la base de datos:

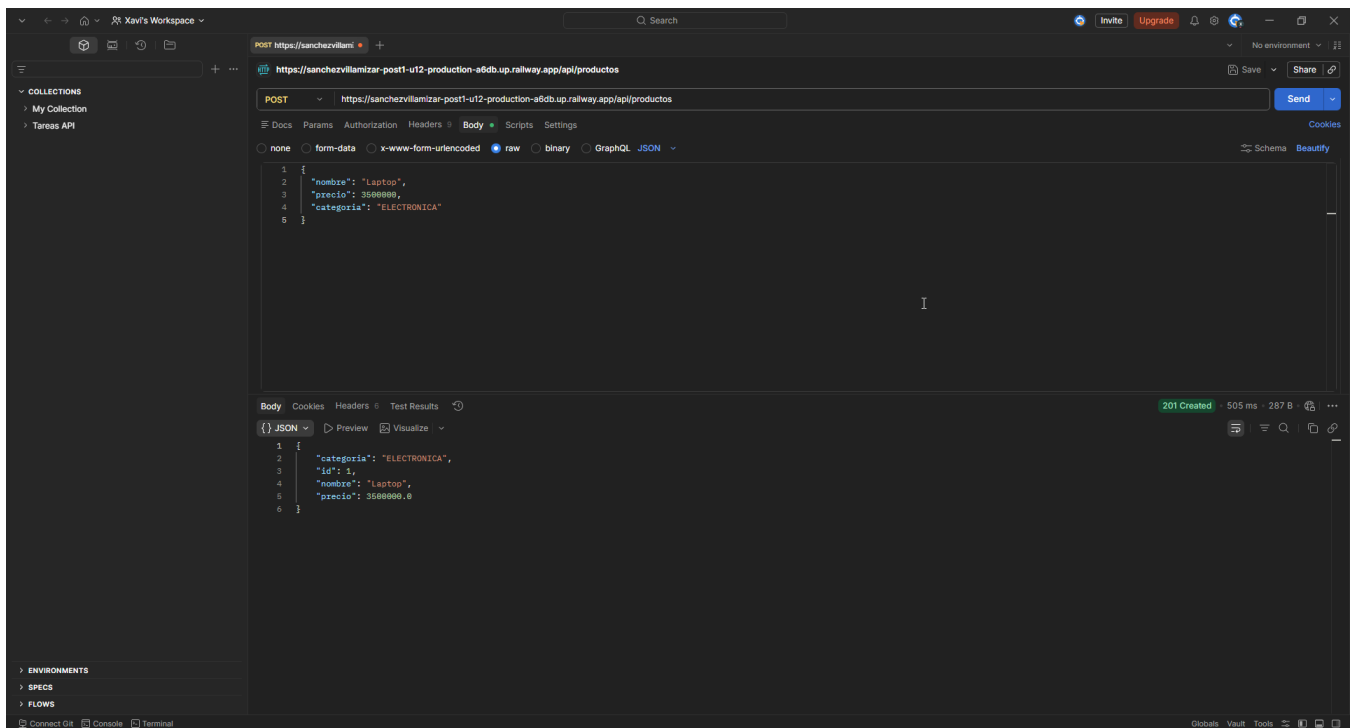
```
POST https://sanchezvillamizar-post1-u12-production-a6db.up.railway.app/api/productos
```

Request Body

```
{ "nombre": "Laptop HP ProBook", "precio": 3500000, "categoria": "ELECTRONICA" }
```

Response (201 Created)

```
{ "id": 1, "nombre": "Laptop HP ProBook", "precio": 3500000.0, "categoria": "ELECTRONICA" }
```



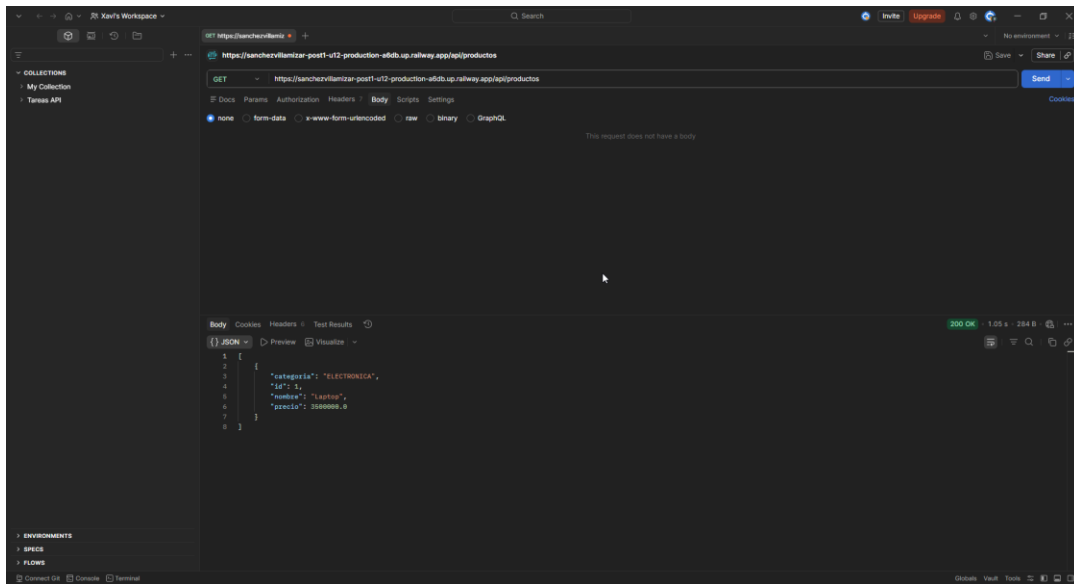
Endpoint 3 — GET /api/productos/{id}

Busca un producto específico por su ID:

```
GET https://sanchezvillamizar-post1-u12-production-a6db.up.railway.app/api/productos/1
```

Response (200 OK)

```
{"id":1,"nombre":"Laptop HP ProBook","precio":3500000.0,"categoria":"ELECTRONICA"}
```

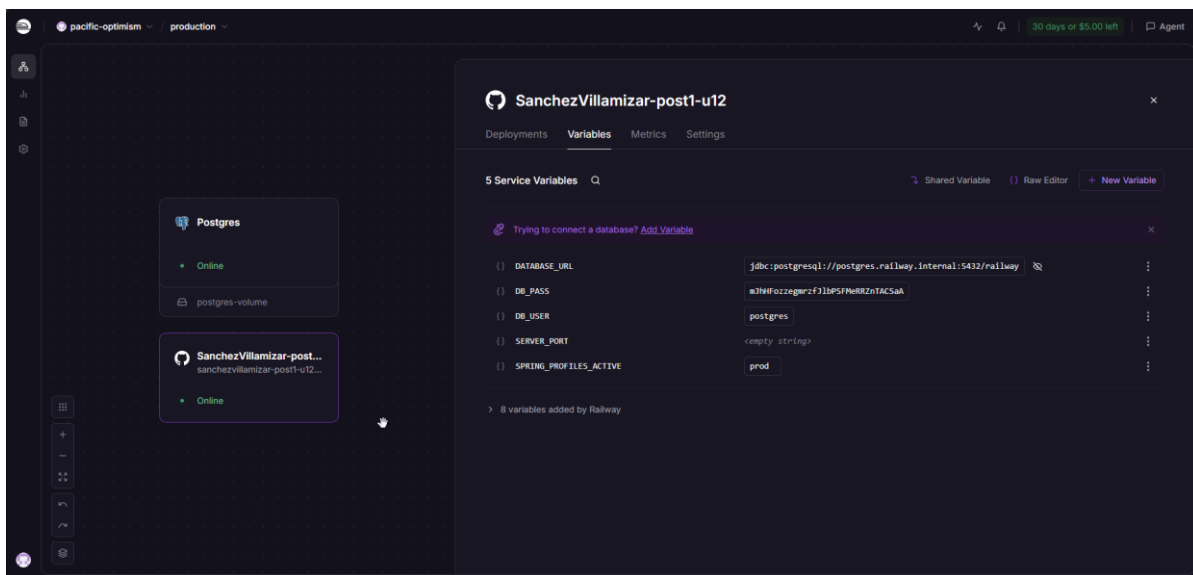


5. Configuración de Variables de Entorno en Railway

Railway permite configurar variables de entorno desde el panel web del servicio sin necesidad de modificar el código fuente. A continuación se describe el proceso seguido y las variables configuradas.

5.1 Proceso de configuración

1. En el panel del proyecto Railway, se seleccionó el servicio de la aplicación Spring Boot.
2. Se navegó a la pestaña Variables del servicio.
3. Se agregaron las variables de entorno necesarias para el perfil de producción.
4. Al guardar, Railway redespoleó automáticamente la aplicación con las nuevas variables.



5.2 Variables configuradas

Variable	Valor configurado	Descripcion
SPRING_PROFILES_ACTIVE	prod	Activa el perfil de produccion
DATABASE_URL	jdbc:postgresql://postgres.railway.internal:5432/railway	URL JDBC de PostgreSQL interno de Railway
DB_USER	postgres	Usuario de la base de datos
DB_PASS	(valor de PGPASSWORD)	Contraseña de la base de datos
SERVER_PORT	\${PORT:8081}	Puerto dinamico asignado por Railway

5.3 Desafios encontrados y soluciones

Durante la configuracion se encontraron dos desafios principales:

- **Formato de DATABASE_URL:** La variable DATABASE_URL de Railway viene en formato postgresql://... (sin el prefijo jdbc:). Spring Boot requiere el formato jdbc:postgresql://... por lo que se configuro el valor completo de forma manual con el prefijo correcto.
- **Puerto dinamico:** Railway asigna el puerto de la aplicacion dinamicamente mediante la variable PORT. Se configuro server.port=\${PORT:8081} en application-prod.properties para que la app use el puerto que Railway asigne.

6. Commits del Repositorio

#	Commit	Descripcion
1	feat: agregar Dockerfile multi-stage y .dockerignore	Imagen de dos etapas: JDK para compilar y JRE para produccion con usuario no root
2	feat: agregar perfiles de produccion, PostgreSQL y docker-compose	application-prod.properties y docker-compose con PostgreSQL y healthcheck
3	fix: configurar variables de entorno y puerto dinamico para Railway	Ajustes de DATABASE_URL y SERVER_PORT para despliegue exitoso en Railway